

Carpool API (Version 0.9.0)

Prerequisites

Place the following files in the `libs/` folder of your project:

- `carpool-X.X.X.jar`
- `ge-core-utils-X-X-X.jar`

These files are local libraries required for the proper functioning of the Carpool system as well as its internal utilities.

Configuration in `build.gradle.kts`

Then add the following dependencies to your Gradle file:

```
dependencies {  
    // Carpool  
    implementation(files("libs/carpool-X.X.X.jar"))  
    implementation(files("libs/ge-core-utils-X.X.X.jar"))  
  
    // OkHttp HTTP client used for network requests (geocoding, APIs like Tavily, etc.)  
    // Cross-platform: Android + Desktop JVM  
    implementation("com.squareup.okhttp3:okhttp:4.12.0")  
  
    // OpenAI  
    implementation("com.openai:openai-java:4.31.0")  
  
    // JSON - kotlinx.serialization  
    implementation("org.jetbrains.kotlinx:kotlinx-serialization-json:1.7.3")  
}
```

Example

```
import java.util.List;
import static carpool.Carpool.check;

public class TestCarpool {
    static String LlmApiKey = " ";
    static String searchApiKey = " ";

    static carpool.PredictionDto prediction = null;

    public static void main(String[] args) {
        carpool.CarpoolClient carpoolClient = carpool.Carpool.create();

        carpoolClient.setLLMProvider(carpool.LLMProvider.OPENAI);
        check(carpoolClient.setLLMApiKey(LlmApiKey));

        carpoolClient.setSearchProvider(carpool.SearchProvider.TAVILY);
        check(carpoolClient.setSearchApiKey(searchApiKey));

        check(carpoolClient.setCenter(
            "ANALAKELY",
            "ANTANANARIVO",
            "" // NOTE: Currently, the country is ignored; everything will work locally in this version because Tavily
operates with local time settings. Leave this field empty.
        ));

        carpool.CarpoolScenario scenario = new carpool.CarpoolScenario(
            2026, 5, 11,
            2026, 5, 12
        );

        var predictionResult = carpoolClient.predict(scenario);
        check(predictionResult);
        if (predictionResult.isSuccess()) {
            prediction = predictionResult.getValueOrThrow();

            System.out.println("Prediction");
            System.out.println("=====");

            List<carpool.ZoneEntryDto> zoneEntries = prediction.getZoneEntries();
            for (int zoneIndex = 0; zoneIndex < zoneEntries.size(); zoneIndex++) {
                carpool.ZoneEntryDto zone = zoneEntries.get(zoneIndex);

                System.out.println();
                System.out.println("Zone #" + zoneIndex);
                System.out.println("    Location          : " + zone.getLocation());
                System.out.println("    Latitude          : " + zone.getLatitude());
                System.out.println("    Longitude         : " + zone.getLongitude());
                System.out.println("    Radius Min Meters : " + zone.getRadiusMinMeters());
                System.out.println("    Radius Max Meters : " + zone.getRadiusMaxMeters());
                System.out.println("    Zone Type         : " + zone.getZoneType());
                System.out.println("    Activity Description: " + zone.getActivityDescription());
                System.out.println("    Note              : " + zone.getNote());

                List<carpool.DateEntryDto> dateEntries = zone.getDateEntries();
                for (int dateIndex = 0; dateIndex < dateEntries.size(); dateIndex++) {
                    carpool.DateEntryDto dateEntry = dateEntries.get(dateIndex);

                    System.out.println();
                    System.out.println("    Date #" + dateIndex);
                    System.out.println("    Date: " + dateEntry.getDate());

                    List<carpool.HourEntryDto> hours = carpool.PredictionDtoKt.allHourValues(dateEntry);
                    for (int hour = 0; hour < hours.size(); hour++) {
                        carpool.HourEntryDto hourEntry = hours.get(hour);
                        System.out.printf(
                            "        Hour %02d: demandLevel=%d, crowdDensity=%d, ridesPerHour=%d\n",
                            hour,
                            hourEntry.getDemandLevel(),
                            hourEntry.getCrowdDensity(),
                            hourEntry.getRidesPerHour()
                        );
                    }
                }
            }
        }
    }
}
```

Sortie

Prediction
=====

Zone #0

Location : Analakely, Antananarivo, Analamanga, Madagascar
Latitude : -18.9086
Longitude : 47.5242
Radius Min Meters : 300.0
Radius Max Meters : 800.0
Zone Type : COMMERCIAL
Activity Description: Daily market shopping, street-food circuit, vendor activity, after-school snack rush
Note : May market activity highlighted around Analakely; strongest stated signal is around 16:00 when school kids arrive for mofo gasy. Free to enter; paid food/shopping; carry small bills and expect bargaining. Estimates reflect likely ride-hailing demand around market arrivals, shopping, school/worker movement, and evening departures.

Date #0

Date: 2026-05-11
Hour 00: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 01: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 02: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 03: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 04: demandLevel=0, crowdDensity=0, ridesPerHour=1
Hour 05: demandLevel=2, crowdDensity=2, ridesPerHour=8
Hour 06: demandLevel=3, crowdDensity=3, ridesPerHour=18
Hour 07: demandLevel=4, crowdDensity=4, ridesPerHour=40
Hour 08: demandLevel=4, crowdDensity=5, ridesPerHour=55
Hour 09: demandLevel=4, crowdDensity=5, ridesPerHour=60
Hour 10: demandLevel=4, crowdDensity=5, ridesPerHour=65
Hour 11: demandLevel=4, crowdDensity=5, ridesPerHour=70
Hour 12: demandLevel=4, crowdDensity=5, ridesPerHour=75
Hour 13: demandLevel=3, crowdDensity=4, ridesPerHour=50
Hour 14: demandLevel=3, crowdDensity=4, ridesPerHour=45
Hour 15: demandLevel=4, crowdDensity=4, ridesPerHour=55
Hour 16: demandLevel=5, crowdDensity=5, ridesPerHour=90
Hour 17: demandLevel=5, crowdDensity=5, ridesPerHour=100
Hour 18: demandLevel=4, crowdDensity=4, ridesPerHour=70
Hour 19: demandLevel=3, crowdDensity=3, ridesPerHour=35
Hour 20: demandLevel=2, crowdDensity=2, ridesPerHour=18
Hour 21: demandLevel=1, crowdDensity=1, ridesPerHour=8
Hour 22: demandLevel=1, crowdDensity=1, ridesPerHour=5
Hour 23: demandLevel=1, crowdDensity=0, ridesPerHour=2

Date #1

Date: 2026-05-12
Hour 00: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 01: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 02: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 03: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 04: demandLevel=0, crowdDensity=0, ridesPerHour=1
Hour 05: demandLevel=2, crowdDensity=2, ridesPerHour=8
Hour 06: demandLevel=3, crowdDensity=3, ridesPerHour=18
Hour 07: demandLevel=4, crowdDensity=4, ridesPerHour=40
Hour 08: demandLevel=4, crowdDensity=5, ridesPerHour=55
Hour 09: demandLevel=4, crowdDensity=5, ridesPerHour=60
Hour 10: demandLevel=4, crowdDensity=5, ridesPerHour=65
Hour 11: demandLevel=4, crowdDensity=5, ridesPerHour=70
Hour 12: demandLevel=4, crowdDensity=5, ridesPerHour=75
Hour 13: demandLevel=3, crowdDensity=4, ridesPerHour=50
Hour 14: demandLevel=3, crowdDensity=4, ridesPerHour=45
Hour 15: demandLevel=4, crowdDensity=4, ridesPerHour=55
Hour 16: demandLevel=5, crowdDensity=5, ridesPerHour=90
Hour 17: demandLevel=5, crowdDensity=5, ridesPerHour=100
Hour 18: demandLevel=4, crowdDensity=4, ridesPerHour=70
Hour 19: demandLevel=3, crowdDensity=3, ridesPerHour=35
Hour 20: demandLevel=2, crowdDensity=2, ridesPerHour=18
Hour 21: demandLevel=1, crowdDensity=1, ridesPerHour=8
Hour 22: demandLevel=1, crowdDensity=1, ridesPerHour=5
Hour 23: demandLevel=1, crowdDensity=0, ridesPerHour=2

Zone #1

Location : Avenue de l'Indépendance, Analakely, Antananarivo, Madagascar
Latitude : -18.9049
Longitude : 47.5209
Radius Min Meters : 400.0
Radius Max Meters : 1200.0
Zone Type : COMMERCIAL
Activity Description: Central business district movement, shops, vendors, taxis, commercial workers, pedestrian traffic
Note : Analakely is described as a dense, walkable city centre with markets, shops, vendors, students, commercial workers, taxi-be links, and taxis. Demand is expected to peak during commute periods, lunch movement, and late afternoon market / shopping exits. Taxi-be minibuses commonly stop around 20:00, which can shift some late trips toward taxi or ride-hailing.

Date #0

Date: 2026-05-11
Hour 00: demandLevel=1, crowdDensity=0, ridesPerHour=2
Hour 01: demandLevel=0, crowdDensity=0, ridesPerHour=1
Hour 02: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 03: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 04: demandLevel=1, crowdDensity=0, ridesPerHour=2
Hour 05: demandLevel=2, crowdDensity=1, ridesPerHour=8
Hour 06: demandLevel=3, crowdDensity=2, ridesPerHour=20
Hour 07: demandLevel=4, crowdDensity=4, ridesPerHour=50
Hour 08: demandLevel=4, crowdDensity=5, ridesPerHour=65
Hour 09: demandLevel=4, crowdDensity=5, ridesPerHour=60
Hour 10: demandLevel=4, crowdDensity=4, ridesPerHour=55
Hour 11: demandLevel=4, crowdDensity=4, ridesPerHour=58
Hour 12: demandLevel=4, crowdDensity=5, ridesPerHour=70
Hour 13: demandLevel=3, crowdDensity=4, ridesPerHour=45
Hour 14: demandLevel=3, crowdDensity=4, ridesPerHour=40

Hour 15: demandLevel=3, crowdDensity=4, ridesPerHour=45
Hour 16: demandLevel=4, crowdDensity=5, ridesPerHour=65
Hour 17: demandLevel=5, crowdDensity=5, ridesPerHour=85
Hour 18: demandLevel=4, crowdDensity=4, ridesPerHour=70
Hour 19: demandLevel=3, crowdDensity=3, ridesPerHour=40
Hour 20: demandLevel=3, crowdDensity=2, ridesPerHour=30
Hour 21: demandLevel=2, crowdDensity=1, ridesPerHour=15
Hour 22: demandLevel=1, crowdDensity=1, ridesPerHour=8
Hour 23: demandLevel=1, crowdDensity=0, ridesPerHour=4

Date #1

Date: 2026-05-12

Hour 00: demandLevel=1, crowdDensity=0, ridesPerHour=2
Hour 01: demandLevel=0, crowdDensity=0, ridesPerHour=1
Hour 02: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 03: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 04: demandLevel=1, crowdDensity=0, ridesPerHour=2
Hour 05: demandLevel=2, crowdDensity=1, ridesPerHour=8
Hour 06: demandLevel=3, crowdDensity=2, ridesPerHour=20
Hour 07: demandLevel=4, crowdDensity=4, ridesPerHour=50
Hour 08: demandLevel=4, crowdDensity=5, ridesPerHour=65
Hour 09: demandLevel=4, crowdDensity=5, ridesPerHour=60
Hour 10: demandLevel=4, crowdDensity=4, ridesPerHour=55
Hour 11: demandLevel=4, crowdDensity=4, ridesPerHour=58
Hour 12: demandLevel=4, crowdDensity=5, ridesPerHour=70
Hour 13: demandLevel=3, crowdDensity=4, ridesPerHour=45
Hour 14: demandLevel=3, crowdDensity=4, ridesPerHour=40
Hour 15: demandLevel=3, crowdDensity=4, ridesPerHour=45
Hour 16: demandLevel=4, crowdDensity=5, ridesPerHour=65
Hour 17: demandLevel=5, crowdDensity=5, ridesPerHour=85
Hour 18: demandLevel=4, crowdDensity=4, ridesPerHour=70
Hour 19: demandLevel=3, crowdDensity=3, ridesPerHour=40
Hour 20: demandLevel=3, crowdDensity=2, ridesPerHour=30
Hour 21: demandLevel=2, crowdDensity=1, ridesPerHour=15
Hour 22: demandLevel=1, crowdDensity=1, ridesPerHour=8
Hour 23: demandLevel=1, crowdDensity=0, ridesPerHour=4

Zone #2

Location : Isoraka, Antananarivo, Madagascar
Latitude : -18.9134
Longitude : 47.5206
Radius Min Meters : 300.0
Radius Max Meters : 900.0
Zone Type : SOCIAL

Activity Description: Restaurants, bars, guesthouses, evening leisure, nightlife

Note : Nearby Isoraka is identified as a central, walkable, lively area with restaurants and nightlife. Demand is most relevant for lunch, dinner, bar arrivals/departures, and post-20:00 trips when taxi-be options reduce. Estimates are moderate because no specific dated event was provided for 2026-05-11 or 2026-05-12.

Date #0

Date: 2026-05-11

Hour 00: demandLevel=2, crowdDensity=2, ridesPerHour=10
Hour 01: demandLevel=1, crowdDensity=1, ridesPerHour=5
Hour 02: demandLevel=1, crowdDensity=0, ridesPerHour=2
Hour 03: demandLevel=0, crowdDensity=0, ridesPerHour=1
Hour 04: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 05: demandLevel=1, crowdDensity=0, ridesPerHour=2
Hour 06: demandLevel=2, crowdDensity=1, ridesPerHour=6
Hour 07: demandLevel=2, crowdDensity=2, ridesPerHour=12
Hour 08: demandLevel=2, crowdDensity=2, ridesPerHour=15
Hour 09: demandLevel=2, crowdDensity=2, ridesPerHour=12
Hour 10: demandLevel=2, crowdDensity=2, ridesPerHour=10
Hour 11: demandLevel=3, crowdDensity=3, ridesPerHour=20
Hour 12: demandLevel=4, crowdDensity=4, ridesPerHour=35
Hour 13: demandLevel=3, crowdDensity=3, ridesPerHour=25
Hour 14: demandLevel=2, crowdDensity=2, ridesPerHour=12
Hour 15: demandLevel=2, crowdDensity=2, ridesPerHour=10
Hour 16: demandLevel=2, crowdDensity=2, ridesPerHour=14
Hour 17: demandLevel=3, crowdDensity=3, ridesPerHour=25
Hour 18: demandLevel=4, crowdDensity=4, ridesPerHour=45
Hour 19: demandLevel=4, crowdDensity=4, ridesPerHour=50
Hour 20: demandLevel=4, crowdDensity=4, ridesPerHour=55
Hour 21: demandLevel=4, crowdDensity=4, ridesPerHour=50
Hour 22: demandLevel=3, crowdDensity=3, ridesPerHour=35
Hour 23: demandLevel=3, crowdDensity=2, ridesPerHour=25

Date #1

Date: 2026-05-12

Hour 00: demandLevel=2, crowdDensity=2, ridesPerHour=10
Hour 01: demandLevel=1, crowdDensity=1, ridesPerHour=5
Hour 02: demandLevel=1, crowdDensity=0, ridesPerHour=2
Hour 03: demandLevel=0, crowdDensity=0, ridesPerHour=1
Hour 04: demandLevel=0, crowdDensity=0, ridesPerHour=0
Hour 05: demandLevel=1, crowdDensity=0, ridesPerHour=2
Hour 06: demandLevel=2, crowdDensity=1, ridesPerHour=6
Hour 07: demandLevel=2, crowdDensity=2, ridesPerHour=12
Hour 08: demandLevel=2, crowdDensity=2, ridesPerHour=15
Hour 09: demandLevel=2, crowdDensity=2, ridesPerHour=12
Hour 10: demandLevel=2, crowdDensity=2, ridesPerHour=10
Hour 11: demandLevel=3, crowdDensity=3, ridesPerHour=20
Hour 12: demandLevel=4, crowdDensity=4, ridesPerHour=35
Hour 13: demandLevel=3, crowdDensity=3, ridesPerHour=25
Hour 14: demandLevel=2, crowdDensity=2, ridesPerHour=12
Hour 15: demandLevel=2, crowdDensity=2, ridesPerHour=10
Hour 16: demandLevel=2, crowdDensity=2, ridesPerHour=14
Hour 17: demandLevel=3, crowdDensity=3, ridesPerHour=25
Hour 18: demandLevel=4, crowdDensity=4, ridesPerHour=45
Hour 19: demandLevel=4, crowdDensity=4, ridesPerHour=50
Hour 20: demandLevel=4, crowdDensity=4, ridesPerHour=55
Hour 21: demandLevel=4, crowdDensity=4, ridesPerHour=50
Hour 22: demandLevel=3, crowdDensity=3, ridesPerHour=35
Hour 23: demandLevel=3, crowdDensity=2, ridesPerHour=25

Meaning of the time-based indicators

demandLevel:

- 0 = no demand
- 1 = very low
- 2 = low
- 3 = medium
- 4 = high
- 5 = very high

crowdDensity:

- 0 = empty
- 1 = very sparsely crowded
- 2 = lightly crowded
- 3 = moderately crowded
- 4 = very crowded
- 5 = extremely crowded

ridesPerHour: estimated number of rides per hour in the area (no fixed limit; free value depending on estimated demand)

Classification of zone types for categorizing locations.

```
@Serializable
enum class ZoneTypeDto(
    val code: Int,
    val key: String
) {
    OTHER (code = 0, key = "other"), // ? Uncategorized or unknown places

    // Sort alphabetically.
    COMMERCIAL (code = 10, key = "commercial"), // 🛒 Shops, malls, business areas
    EDUCATION (code = 20, key = "education"), // 🎓 Schools, universities, training centers
    EVENT (code = 30, key = "event"), // 🎉 Concerts, festivals, special events
    GOVERNMENT_SERVICE (code = 40, key = "government_service"), // 🏛️ Government offices & public administration
    HEALTHCARE (code = 50, key = "healthcare"), // 🏥 Hospitals, clinics, pharmacies
    RESIDENTIAL (code = 60, key = "residential"), // 🏠 Homes, apartments, neighborhoods
    SERVICE (code = 70, key = "service"), // 🛠️ Daily services & utilities
    SOCIAL (code = 80, key = "social"), // 🍷 Restaurants, cafés, meeting places
    TOURISM (code = 90, key = "tourism"), // 🌐 Hotels, attractions, tourist areas
    TRANSPORT_HUB (code = 100, key = "transport_hub"), // 🚉 Major transport & transit locations
    WORK (code = 110, key = "work"), // 🏢 Offices, companies, workplaces

    companion object {
        fun serializer(): KSerializer<ZoneTypeDto> {
        }
    }
}
```

‘check’ function

The check function is designed to display an error dialog when an operation fails.

* Without check

```
carpoolClient.setLLMApiKey("");
```

In this case, only an error is displayed in the logs. If you forget to use `check`, a message will still be recorded in the logs, but the program will not stop.

Error in 'setLLMApiKey': apiKey is required - at TestCarpool.java:10

*With check

```
check(carpoolClient.setLLMApiKey(""));
```

In this case, in addition to the error being logged, an error dialog box is also displayed. It is recommended to always use `check`, as it makes errors much more visible.

Error in 'setLLMApiKey': apiKey is required - at TestCarpool.java:10

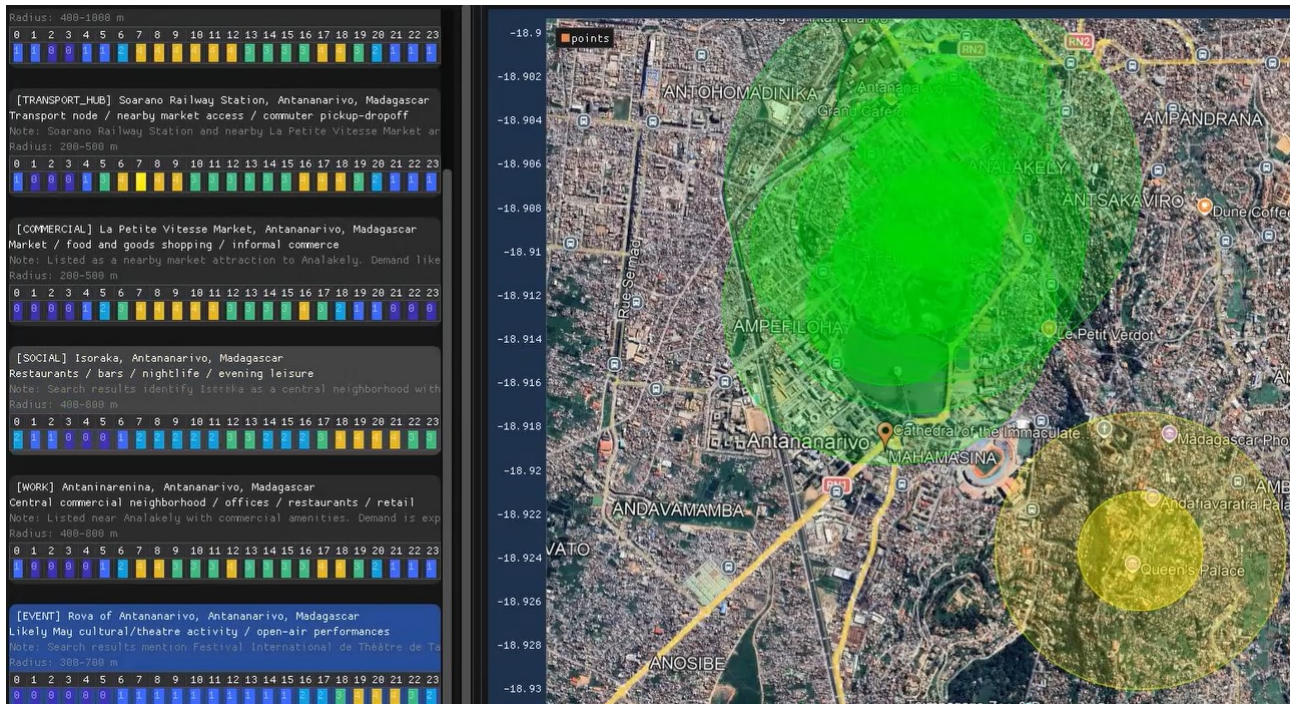


Example: Safe usage of CarpoolResult with getValueOrThrow()

```
CarpoolResult<PredictionDto> result = ...;
check(result);
if (result.isSuccess()) {
    PredictionDto prediction = result.getValueOrThrow();
    // * ⚠: Calling getValueOrThrow() without checking success may
    // cause a crash.
    Prediction.[...]; // use prediction safely here
}
```

Example of integration into a UI

Note: To retrieve data across the entire city, you need to use a loop over the locations and call the ‘predict’ function multiple times.



Interface (Kotlin)

```
interface CarpoolClient {  
  
    //-----  
    // API Configuration  
    //-----  
  
    /**  
     * Returns the currently active LLM provider.  
     */  
    fun getLLMProvider(): LLMProvider  
  
    /**  
     * Sets the LLM provider (e.g. OpenAI).  
     *  
     * @param provider selected provider  
     */  
    fun setLLMProvider(provider: LLMProvider)  
  
    /**  
     * Returns the configured LLM API key.  
     *  
     * @return empty string if not set  
     */  
    fun getLLMApiKey(): String  
  
    /**  
     * Sets the API key used for LLM calls.  
     *  
     * Errors:  
     * - [CarpoolStatus.InvalidParameter] → apiKey is blank  
     */  
    fun setLLMApiKey(apiKey: String): CarpoolResult<Unit>  
  
    /**  
     * Returns active search provider (e.g. Tavily).  
     */  
}
```

```

*/
fun getSearchProvider(): SearchProvider

/**
 * Sets search provider.
 */
fun setSearchProvider(provider: SearchProvider)

/**
 * Returns configured search API key.
 */
fun getSearchApiKey(): String

/**
 * Sets search API key.
 *
 * Errors:
 * - [CarpoolStatus.InvalidParameter] → apiKey is blank
 */
fun setSearchApiKey(apiKey: String): CarpoolResult<Unit>

//-----
// Center Configuration
//-----

/**
 * Sets the active geographic center used for prediction.
 *
 * This defines the spatial anchor for:
 * - search queries
 * - event extraction
 * - prediction zones
 *
 * Errors:
 * - [CarpoolStatus.InvalidParameter] → [districtName] is blank
 * - [CarpoolStatus.InvalidParameter] → [cityName] is blank
 */
fun setCenter(districtName: String, cityName: String, countryName: String): CarpoolResult<Unit>

//-----
// Prediction
//-----

/**
 * Runs the prediction process and returns structured results.
 *
 * The process involves:
 * - validating configuration
 * - collecting external information
 * - processing data using AI models
 * - producing structured prediction output
 *
 * ##### Errors (validation phase):
 * - [CarpoolStatus.InvalidParameter] → missing LLM provider
 * - [CarpoolStatus.InvalidParameter] → missing search provider
 * - [CarpoolStatus.InvalidParameter] → missing API keys
 * - [CarpoolStatus.InvalidParameter] → no center configured
 * - [CarpoolStatus.InvalidParameter] → invalid date range
 *
 * ##### Errors (runtime phase):
 * - [CarpoolStatus.NetworkUnavailable] → search/LLM network failure
 * - [CarpoolStatus.NetworkRequestTimedOut] → request timeout
 * - [CarpoolStatus.InvalidLlmApiKey] → LLM rejected API key
 * - [CarpoolStatus.InvalidSearchApiKey] → search rejected API key
 * - [CarpoolStatus.UnknownError] → unexpected internal failure
 *
 * ##### Errors (AI output phase):
 * - [CarpoolStatus.UnknownError] → AI returned an empty response

```



```

* - [CarpoolStatus.UnknownError] → AI returned malformed JSON
*/
fun predict(scenario: CarpoolScenario): CarpoolResult<PredictionDto>

//-----
// Coordinate Conversion & Geocoding Utilities
//
// NOTE:
// These are part of the public API surface of this interface.
// Geocoding may in the future use external providers (e.g. Google Maps, AI-based
// services) and may require API keys, so it is treated as a configurable capability.
// -----

/**
 * Converts DMS (Degrees/Minutes/Seconds) into decimal coordinates.
 *
 * E.g. (0°0'0.00"N, 0°0'0.00"E) --> (0, 0, 0.0, 'N', 0, 0, 0.0, 'E')
 *
 * Errors:
 * - [CarpoolStatus.UnknownError] → invalid numeric conversion or unexpected failure
 */
fun coordinatesFromDMS(
    latDeg: Int, latMin: Int, latSec: Double, latDir: Char, // 0°0'0.00"N --> 0, 0, 0.0, 'N'
    lonDeg: Int, lonMin: Int, lonSec: Double, lonDir: Char // 0°0'0.00"E --> 0, 0, 0.0, 'E'
): CarpoolResult<LatLonDto>

/**
 * Converts a human-readable location into coordinates using geocoding.
 *
 * This is an optional method used instead of AI-provided coordinates.
 * Nominatim may fail to resolve some locations, but provides high-precision
 * results when successful.
 *
 * Errors:
 * - [CarpoolStatus.NoResultsFound] → location not found
 * - [CarpoolStatus.NetworkUnavailable] → no internet connection
 * - [CarpoolStatus.NetworkRequestTimedOut] → request timeout
 * - [CarpoolStatus.UnknownError] → unexpected geocoding failure
 */
fun coordinatesFromLocationUsingGeocoding(location: String): CarpoolResult<LatLonDto>

/**
 * Converts meters into latitude degree offset.
 *
 * @param latitudeDeg reference latitude used to approximate Earth curvature at that position
 *
 * ##### Note:
 * - approximation based on WGS84 ellipsoid
 */
fun metersToLatDegrees(meters: Double, latitudeDeg: Double): Double

/**
 * Converts meters into longitude degree offset.
 *
 * @param latitudeDeg reference latitude used because longitude distance varies with latitude
 *
 * ##### Note:
 * - depends on latitude due to Earth's curvature
 */
fun metersToLonDegrees(meters: Double, latitudeDeg: Double): Double
}

```